

Git & Collaboration Technical Assessment Report

Detailed report for [vinay.mandloi.averybit@gmail.com](#)'s attempt on May 21, 2026, 10:30 a.m..

Final Result: Fail

You did not meet the required cutoff score for this assessment.



Shortlist

Not Shortlisted

YOUR SCORE

0%

CUTOFF SCORE

70%

CORRECT
ANSWERS

0 / 10

INCORRECT
ANSWERS

9 / 10

Assessment Integrity

AI PROCTORED

TRUST SCORE



100% Trust

High trust level. No significant suspicious activity detected.

SECURITY TIMELINE

Periodic Identity Check Periodic proctoring snapshot Snapshot	10:31:31
Periodic Identity Check Periodic proctoring snapshot Snapshot	10:32:01
Periodic Identity Check Periodic proctoring snapshot Snapshot	10:32:31
Periodic Identity Check Periodic proctoring snapshot Snapshot	10:33:01

Questions & Answers

1. What is the difference between git pull and git fetch? When would you prefer one over the other?

Incorrect

Your Answer:

Git pull = git pull is the the process of git hub and its is the main process of git hub git pull is work is develop the code in branch and git pull is store the code in branch git fetch= is the also part of git hub is work is

Correct Answer:

git fetch downloads changes from the remote but does not apply them to your working branch. Your local code stays untouched.git pull = git fetch + git merge (or rebase). It fetches and immediately integrates changes into your current branch.Prefer fetch when you want to inspect changes before merging. Use pull when you're confident and want a quick sync.Key line: git fetch is safe and non-destructive. git pull can cause merge conflicts immediately.

2. What does git stash do? Give a real-world scenario where you'd use it.

Incorrect

Your Answer:

git stash is the process of the developers code are copies and safe of main repo and developer added the feature .

Correct Answer:

git stash temporarily shelves (saves) your uncommitted changes — both staged and unstaged — so your working directory is clean.Use git stash pop to restore them later.Real-world scenario: You're mid-feature and a urgent bug report comes in. You stash your work-in-progress, switch to main, fix the bug, then come back and pop your stash.git stash ← saves changesgit checkout main... fix bug ...git stash pop ← restores changes

3. What is the difference between git reset --soft, --mixed, and --hard? If you ran git reset --hard HEAD~1 by mistake, how would you recover the lost commit?

Incorrect

Your Answer:

Not Attempted

Correct Answer:

--soft — moves HEAD back but keeps changes staged. Commit is undone, files are ready to re-commit.--mixed (default) — moves HEAD back, un-stages changes, but keeps them in the working directory.--hard — moves HEAD back and destroys all changes in staging and working directory. Dangerous.Recovery after accidental --hard: use git reflog — it logs every position HEAD has been at. Find the lost commit's hash and run git checkout or git reset --hard .git reflog ← find the commit hashgit reset --hard abc1234 ← restore to it

4. Describe a merge conflict scenario in detail. How do you resolve it? What do the conflict markers (<<<<<<, =====, >>>>>>) represent?

Incorrect

Your Answer:

merge conflict is the two developer commit the same branch at a time and git hub is not declare the how has write so this is solution is first of all visit a branch and set the manually fix the exact write commit and and command is git location git branch and status of branch git status and delete the extra commit it has the solution apply to the merge conflict .

Correct Answer:

A merge conflict occurs when two branches modify the same line(s) of a file differently and Git cannot auto-resolve which version to keep.Conflict markers explained:<<<<<< HEADyour current branch's version=====incoming branch's version>>>>>> feature/loginResolution steps: Open the file, manually edit to the correct final version, remove all markers, then run git add and git commit to complete the merge.Tools like VS Code, IntelliJ, or git mergetool provide a 3-way diff UI to make this easier.

5. What is git bisect and how does it work? Describe the steps to use it to find which commit introduced a bug.

Incorrect

Your Answer:

git bisect is the basically

Correct Answer:

search space each step. Steps: git bisect start
git bisect bad ← current commit is broken
git bisect good v1.0 ← this old commit was fine... Git checks out a middle commit test it manually ...
git bisect good (or bad) ← repeat until done
git bisect reset ← return to original HEAD
Git narrows it down to a single commit — the exact one that broke things. For 1000 commits, it takes only ~10 steps.

6. Explain the three areas of a Git project: the working directory, staging area (index), and repository. How does a file move between them?

Incorrect

Your Answer:

1 = working directory is the area of work and implement the code of developers . 2 = staging area is also area of git like (index) it is work is index the the every fit data like repo and branches 3 = repository is like a data base developer store their code in repository and secure the data and also copy the data .

Correct Answer:

Working directory — where you edit files. Changes here are "untracked" or "modified". Staging area (index) — a prep zone. git add moves changes here. Only staged changes go into the next commit. Repository (.git) — permanent history. git commit takes everything staged and saves it as a snapshot. Edit file → git add → (staging) → git commit → (repository)

7. What is the purpose of a .gitignore file? Give three examples of files that should typically be ignored in a Node.js or Python project.

Incorrect

Your Answer:

gitignore is the basically in the process of ignore the repo .

Correct Answer:

.gitignore tells git which files and folders to never track or commit — keeps the repo clean of generated/sensitive/local files. Node.js examples: node_modules/, .env, *.log Python examples: __pycache__/, .venv/, *.pyc Rule of thumb: ignore anything that is auto-generated, machine-specific, or contains secrets.

8. What is a Pull Request (PR) in GitHub? Describe the typical lifecycle of a PR from creation to merge.

Incorrect

Your Answer:

Pull request is the main part of GitHub because the developers create the branch and merge the project of one URL and generate the (PR) Request . The lifecycle of PR request is first of all 1 = Create a branch git branch 2= git commit -m "message" 3= git checkout -b branch name 4= push the code git push 5= marge the branches 6=Create PR Request

Correct Answer:

A PR is a GitHub mechanism to propose merging changes from one branch into another, with built-in review, discussion, and CI checks.Lifecycle:1. Create a feature branch2. Push branch to GitHub3. Open a PR (base: main ← compare: feature)4. Reviewers comment, request changes5. Author pushes fixes (commits update the PR automatically)6. Approvals received + CI passes7. Merge (squash / merge commit / rebase)8. Feature branch deleted

9. Explain the difference between git merge and git rebase. What are the trade-offs of each?

Incorrect

Your Answer:

1 = git merge is a part of git hub and there are the work on marge the branches and create the project in cloud base platform of git hub and the one of the most and every developer are chase the merge conflict is like bug of the system 2 = rebase is the part of git hub and the work of rebase is clone the project .

Correct Answer:

Merge — creates a new "merge commit" that joins two branch histories. Preserves the full history exactly as it happened. Safe for shared branches.Rebase — re-writes your commits on top of the target branch, producing a linear history. Cleaner log, but rewrites commit hashes.Golden rule: never rebase a branch others are working on. Rebase your local feature branch before opening a PR; merge when integrating into main.Merge = safe + preserves history. Rebase = clean + rewrites history.

10. You accidentally committed sensitive credentials (e.g., an API key) and pushed to a public GitHub repo. Walk through the exact steps you would take to remediate this, and why simply deleting the commit locally is not enough.

Incorrect

Your Answer:

The problem is sensitive credentials an API key and pushed git hub repo so it is not simply deleting the repo because the git is the version control system and git hub is the cloud base . so git control the git hub to over system the git bash download so command are solve the problem . simple deleting is not enough because the git hub has privacy and policy's so delete the commit as repo as first of all check the location of branch git branch and delete the commit git delete -d .

Correct Answer:

1. Revoke/rotate the secret immediately — invalidate the API key/password on the provider side. This is step 1, not step 4.
2. Remove from history using git filter-repo (preferred) or BFG Repo Cleaner to scrub the secret from every commit.
3. Force-push the cleaned history: git push --force to all branches and tags.
4. Contact GitHub Support to clear cached/forked copies.
Why deleting locally isn't enough: Git history is immutable by design. The commit still exists in the remote, is cached by GitHub, and may already be cloned or indexed by bots — within seconds of a push.